

Watcher v2 — Build Roadmap

From a message filer to a receptionist. Every item scored for urgency and ease. • **v2.6** • 16 August 2026 • supersedes the v2.5 PDF of 15 August

What this revision does. v2.4 put the critical path outside Track A for the first time: host it, then make it safe. Session 8 built the hosting half. **The schema now exists on a real database** (Supabase, eu-central-1, migrated and stamped), **tenant isolation is enforced by Postgres rather than by convention** (RLS forced on all 20 tables, behind a role that cannot bypass it), and **the API is a container image** at the path cd.yml has been waiting on since it was written — and **the API is deployed and serving at** watcher-api-lup7.onrender.com. What is left of Track B is a domain and the durable queue. **Remaining: ~34.25 engineering days.**

Where we stand today

Built and tested	Missing — nothing ships without these
417 passing tests, no DB or network needed 86 source files across 20 modules (125 with tests) 19 DB tables + 4 migrations, applied to a live database Every external system behind a swappable seam Eval runner + CI accuracy gate, 50 golden cases Receptionist vocabulary: 19 intents, data not code Prompt v3 — catalogue rendered from the vocabulary Channel-neutral envelope + chat/voice adapters Task state machine and the autonomy gate Typed config over every .env variable (A1) Anthropic + OpenAI on the LLM seam, caching on (A3) NEW • Engine + session scope, pooler-safe by default (A2) NEW • Composition root: the process starts and files a message end to end (A4) NEW • Conversation continuity: one task carried across turns, classifications recorded (A5) Outbound sender — the reply reaches the guest (A6) NEW • Supabase project, migrated, with the channel_configs row (B1) NEW • RLS forced on every table, per-transaction tenant GUC, cross-tenant test (B2) NEW • Container image; the project installs from pyproject (B3) NEW • Deployed and serving on Render, Frankfurt (B3)	No DB connection — done (A2) No endpoint — done (A4) No memory across turns — done (A5) No outbound — done (A6) No emergency detection — emergency=False is still hardcoded (G3) No slot extraction — the model emits no slots, so a task fills only by handing off (2.x) No database — done (B1) No RLS — done (B2) No Dockerfile — done (B3) No outbound credentials — the deployed process warns at startup: it composes and records replies it cannot deliver The database path is unproven — SQLAlchemy connects lazily, so the pooler host is tested by the first message, not by startup No custom domain, no webhook subscription — nothing routes a guest to it yet (B4) No control page — one CSS file, no package.json No control-page API — ~25 endpoints, none written Knowledge — zero tables, zero rows Live availability — no read path to any PMS A measured prompt — the gate replays v2 fixtures, so 88% is not v3's number

What changed in this revision (v2.4 → v2.5)

Delivered	Consequences
B1, B2 and B3 delivered — 2.75 engineering days. Total 37 → 34.25 Track B: 4.25 → 1.5 days Tests 406 → 417 (+11). Migration 004 is the first one nothing autogenerated watcher_app — the application connects as a role that cannot bypass RLS; Supabase's postgres role can, which would have made the policies decorative Every tenant-facing adapter now takes a TenantScope rather than a SessionScope	Tenant isolation is a property of the database now, not of remembering the WHERE clause — the claim AGENTS.md calls non-negotiable is enforced and tested Building the image found two defects the pinned test environment hides: the intent vocabulary was missing from the wheel, and Starlette 1.0 removed add_event_handler G3 is now the only thing between here and a number a guest can message — B4 is half a day of DNS once a service exists The service is live — four deploys to get there, three of which failed on the config gate refusing to start without its secrets, which is the gate working Unchanged: 2.4 is still the last item before a demo; 3.1 still blocked on P1; the scope guard still holds — no PDF handbook ingestion

Totals: ~34.25 engineering days remaining. At the observed 2–3 engineering days per working session that is **11–17 sessions**; at the six-day week these estimates assume, ~5.7 weeks of pure build — so plan **~7.5 weeks to sellable** and **~4 days to a real number that answers safely**. The critical path is now **G3 to make it safe, B4 to expose it, and 2.4** — G3 first, because the system already answers.

Track A — Make it run. [COMPLETE — ALL SIX DONE]

Configuration, a database, a model, an endpoint, continuity and a sender. The track that turned a library into a running receptionist is finished; what follows is hosting it safely.

#	Work item	Urgency	Ease	Days	Status
A1	Configuration layer	DONE	Easy	0.5	DONE — core/config.py; every variable in .env.example typed, required per subsystem, placeholders read as absence, secrets as SecretStr
A2	DB engine and session	DONE	Easy	0.5	DONE — db/engine.py; engine, sessionmaker, session scope. Transaction-mode pooling assumed by default (NullPool + prepare_threshold=None), DATABASE_POOL_MODE opts out; a bare postgresql:// URI is rewritten onto psycopg 3; alembic connects the same way
A3	Concrete LLM providers	DONE	Moderate	1.5	DONE — Anthropic + OpenAI on the existing seam; system block marked cacheable; per-call usage reported, so cost is measurable
A4	Composition root	DONE	Moderate	1.0	DONE — apps/api/main.py, run as uvicorn apps.api.main:create_application --factory. Also the five database port implementations create_app needed and a process-level worker pool, so the webhook still answers before classification runs
A5	Wire continuity + excise v1 filer	DONE	Moderate	2.25	DONE — a ConversationStore in the message path: the inbound turn is recorded, the active task is loaded and saved back, the reply joins the transcript. process() is async; classifications is populated with measured latency and the prompt version; the rules/destinations threading is gone (D24), tables retained
A6	Outbound sender	DONE	Moderate	1.0	DONE — WhatsAppSender on the ChannelSender seam, quick replies rendered within the channel's limits, bounded retries. The credential fields moved behind channels/config.py and the last KNOWN_LEAKS entry is gone with them

What Track A leaves behind. Items 2.1 and 2.2 were genuinely written — the tables, the repository, the task state machine, the reply path — and the line joining them never was. A5 wrote that line and A6 wrote the wire. The milestone “holds a conversation” is now true, and the next thing between this and a real guest is not another feature: it is **B1–B4 to host it and G3 to make it safe**.

Track B — Host it. [Supabase + Render]

Stack locked 15 August 2026. B2 must land before a second client exists. B1 grew one precondition this session — see its note.

#	Work item	Urgency	Ease	Days	Status
B1	Supabase project and migrations	DONE	Easy	0	DONE — watcher-prod in eu-central-1, migrations applied and stamped, one enabled channel_configs row. Its external_id is a placeholder until the Meta phone-number id replaces it — one UPDATE, in the deploy runbook
B2	Row-Level Security	DONE	Moderate	0	DONE — migration 004: enabled and forced on all 20 tables, policies on app.current_tenant, and a watcher_app role that cannot bypass them (Supabase's postgres role can, which is the trap). Verified cross-tenant on the live database
B3	Dockerfile and Render service	DONE	Easy	0	DONE — apps/api/Dockerfile, which activates the cd.yml image job, and the service is live in Frankfurt on the free plan. Sending is still degraded until WHATSAPP_ACCESS_TOKEN and WHATSAPP_PHONE_NUMBER_ID are set
B4	Domain, TLS, webhook subscription	HIGH	Easy	0.5	NOT STARTED — the platform needs a public HTTPS URL before anything arrives
B5	Durable queue (Redis + arq)	MED	Moderate	1.0	NOT BUILT — the in-process worker pool A4 ships loses in-flight classifications on every deploy. Persist-before-enqueue means a message is never lost, only its classification. Same seam, same consumer

Runs in parallel — start today. P1 pick the first client (NOW, blocks 3.1) • P2 PMS sandbox keys (HIGH, 0.5d) • P3 file the platform business verification (HIGH, 1 hr) • P4 Graphify as a build aid (LOW, optional). Founder time, excluded from the total — and P2 and P3 are the two that make Track B's week land on time.

Track 2 — Make it a receptionist. [2.4, 2.7, 2.8 REMAIN]

What remains is knowledge, a measured prompt, and more than one property.

#	Work item	Urgency	Ease	Days	Status
2.1	Conversations, tasks and slot filling	HIGH	Hard	spent	WIRED (A5) — 2.0d spent here, the wiring priced in A5. Slot <i>extraction</i> is the one part still missing: the model emits no slots, so a task fills only by the clarifying-turn budget expiring. That is a prompt change and a golden-set change
2.2	The reply path	HIGH	Moderate	spent	WIRED (A6) — 1.5d spent here, the sender priced in A6
2.3	Autonomy gate	DONE	Easy	1.0	DONE — RECEPTIONIST_REPLY as the fourth RoutingAction
2.4	Knowledge base	HIGH	Moderate	2.0	NEXT after Track A — facts table with sensitivity flags, a real "I don't know" that fetches a human, verification codes
2.5	Prompt v2 and rewrite the golden set	DONE	Moderate	1.0	DONE — golden set 8 → 50 across all 19 intents
2.6	Intent taxonomy unification	DONE	Easy	0.5	DONE — 19 vocabulary-aligned intents
2.7	Re-record the eval under prompt v3	MED	Easy	0.5	UNBLOCKED — A3 landed; needs only a key. Add Franco-Arabic cases, and settle the two open numbers on the next page
2.8	Many properties per client	HIGH	Moderate	1.0	properties table, per-property fact scoping, message → property resolution

The eval is still not measuring the prompt. The gate runs and passes, but it replays fixtures keyed by message text and recorded under prompt v2, so it reports 88% whatever the prompt says. Until 2.7 runs, treat 0.88 as v2's number. baseline.json says so in the file, and correctly records claude-haiku-4-5-20251001 as the model that produced those fixtures.

Tracks D, G, E and 3 — unchanged from v2.1

Summarised here; the v2.1 PDF carries the full per-item detail.

D — The control page (D0–D10) 13.75d	Six receptionist views. D2 (REST API, 3.0d) is the hidden half — “build the control page” reads as frontend work; three of its days are backend endpoints that do not exist. Still the item most likely to be cut by accident
G — Receptionist guardrails (G1–G4) 5.5d	Not new features — these enforce rules intents.yaml states and schema.py validates and nothing in apps/ honours. G3 (emergency path, 1.5d) is not optional before a real guest: worker.py hardcodes emergency=False, so a gas leak files a maintenance ticket
E — What makes it sellable (E1–E5) 4.5d	Tenant onboarding, usage metering, billing, observability, backups and residency
3 — Integration and measurement (3.1–3.3) 5.5d	3.1 blocked on P1. Base360.ai ruled out; Hostaway, Guesty and Cloudbeds all publish APIs

New this revision: what A5 and A6 settled, and what they left open

Continuity	Every classified message now runs against a ConversationStore : the inbound turn is recorded, the job already in flight is loaded, and the updated job and the reply are written back. A receptionist constructed without a store is refused — one that forgets the previous turn looks like it works, which is the exact failure this item existed to remove. ConversationRepository, written in item 2.1, is finally called by something other than its own tests
The clarifying-turn budget	Continuity introduces a failure mode that no-continuity does not: a task that cannot make progress no longer fails, it loops . The vocabulary has declared <code>max_clarifying_turns = 3</code> and <code>on_max_turns = handoff_to_human</code> since item 0.3 and nothing read them. They are read now. The guard applies to asking, never to acting: a task with everything it needs still completes on the last turn of the budget
The v1 filer	The orchestrator used to answer an inbound message twice — auto-route it by rule or band, <i>and</i> reply to it. Routing to a spreadsheet was v1's answer; the receptionist is v2's. Rules and destinations are gone from the message path (D24) and both tables are retained , because deliberate routing by a human is a control-page feature and this is the evaluator it will use. AUTO_ROUTE went with them: there is nothing to route to
classifications, at last	The table sat empty for four sessions because two of its columns — <code>latency_ms</code> and <code>prompt_version</code> — had no honest source. The classifier reports them now rather than the writer inventing them: latency spans the whole tiered policy , retries and escalation included, because that is what the guest waited for. <code>model_used</code> , which used to be handed to the inbox writer and dropped, lives on that row, and <code>inbox_items</code> points at it
The wire	WhatsAppSender posts to the versioned Graph endpoint for the number we send as. The three-button cap and the 20-character title limit are applied there and nowhere else, and neither raises. A failed send is logged and reported, never raised — by then the message is classified, the reply recorded and the decision filed, and a transient 502 must not undo all of it. The reply is recorded <i>before</i> it is sent, deliberately
Item 1.1, closed	The channel credential fields moved from <code>core/config.py</code> to <code>channels/config.py</code> , which Settings extends: one object still reads one environment, but the knowledge of what a send needs sits with the channel. KNOWN_LEAKS is now empty — no core file names a channel — and the machinery stays, because an empty allowlist is the strongest form of that test and a phone line is next
Still not written	Slot extraction . The receptionist receives an empty slot dict because ClassificationResult has no slots field; adding one is a prompt change and a golden-set change, which is item 2.x. The consequence is bounded and honest: a task fills only by the budget expiring and then fetches a person. Emergency detection is still hardcoded to False — that is G3, and it is the last safety gap

Two numbers nobody has measured, both cheap to settle during 2.7

1. The prompt's real token count	~5k is a <code>characters÷4</code> estimate. Haiku 4.5 will not cache a prefix below 4,096 tokens , and the cheap tier is where caching pays for itself. The estimate clears the floor by ~30%, and the true count is probably higher (Arabic and Franco-Arabic tokenize worse than the approximation), so caching almost certainly activates — but if the vocabulary shrinks it stops silently: no error, just a larger bill. Sonnet 5's floor is 1,024, so escalation is not exposed either way
2. Cost per message	Still unmeasured, but measurable : TokenUsage reports cached versus fresh input tokens per call, normalised across both providers. Measure before quoting a price

How the total reconciles

A 0 + B 1.5 + D 13.75 + E 4.5 + G 5.5 + (2.4, 2.7, 2.8) 3.5 + (3.1, 3.2, 3.3) 5.5 = **34.25**

Rows 2.1 and 2.2 show “spent” rather than a number because their days are already delivered and their wiring was priced in A5 and A6, now built — adding them would double-count 3.5 days. P1–P4 is excluded: founder time, not engineering. Counting scope loosely is what produced the 5.75 figure that v2.1 replaced, so the arithmetic is stated rather than implied.

The dates

Milestone	v2.5 said	Now	Status
M1 — answers a real message, safely	~4.25 days	~4 days	NEXT — G3 + B4 + 2.4. It is deployed and it answers; the emergency path is what “safely” still depends on
M2 — you can watch and correct it	+13.75 days	+13.75 days	PENDING — Track D, six receptionist views
M3 — sellable	+16.5 days	+16.5 days	PENDING — Tracks E, F and the rest of G. Blocked on P1
Phone answering	after M3	after M3	PENDING — the speech-to-text seam already exists

What would actually move these dates

The risks v2.3 named, re-scored against what session 7 learned.

Supabase connection pooling	Downgraded from imminent to unmeasured. A2 assumes the transaction pooler and is safe there by default. What is not known is the cost of that safety — NullPool trades a handshake per checkout for correctness. Measure it during B1/B3 and flip DATABASE_POOL_MODE if session mode wins; do not settle it in production at 2am
A missing channel_configs row	NEW. The first deployed webhook will 500 on every message until B1 inserts one row per endpoint. It is a one-line fix and a genuinely confusing hour if nobody wrote it down, which is why it is in B1's status, the README and the spec
RLS retrofitted late	Unchanged. Adding row-level security after application queries exist surfaces as silent empty result sets rather than errors. B2 sits before the second tenant for that reason
D2 is invisible in its own track title	Unchanged, and still the most likely accidental cut. Three of Track D's days are backend
Cost per message	Unmeasured, not unknown. A3 made it measurable and turned prompt caching on. Measure before quoting a price
PMS API access	Unchanged. Docs are public, but sandbox approval and rate limits are theirs to grant. Start P2 today
Three roadmap artifacts disagree	CLOSED. docs/make_roadmap.py generates this document into docs/Watcher_v2_Roadmap.pdf. Edit the generator and re-run it; do not hand-write another version
A receptionist that answers but cannot recognise an emergency	NEW, and the most serious item on this page. Until A6 a gas leak was filed in silence; now it gets a polite, confident reply about maintenance. Answering raises the cost of emergency=False rather than lowering it, which is why G3 moves ahead of 2.4 for any deployment a real guest can reach
Two messages arriving at once	NEW, bounded. Two turns in one thread classified concurrently can race and open two conversations. Per-message ordering is a property of the queue, and the queue is an in-process pool until B5 , where ordered per-conversation delivery belongs. Recorded rather than papered over with a lock that would not survive a second process

If you do only one thing next session: B1, then G3. Not 2.4, and not D. The receptionist now answers — on a laptop, to a number nobody can message. B1–B4 is 3.25 days and ends with a public HTTPS URL the platform can deliver to (and B1 must insert the channel_configs row, or every message 500s). **G3 is 1.5 days and it is what makes answering safe:** emergency=False is hardcoded, so a gas leak now gets a confident reply about maintenance rather than a person. Answering made that worse, not better.

Session log

Session	Delivered	Tests	Days
1	PR #13 — items 1.1, 1.3, 2.1, 2.2, 2.3	228 → 248	6.5
2	PR #14 and #15 — items 2.5, 2.6	248 → 259	1.5
3	PR #16 — prompt v3, unplanned	259 → 277	~0.5
4	Code audit plus a design review. No application code changed. Five unpriced tracks added; 2.1/2.2 downgraded to part-built; emergency detection found unwired; the control page rebuilt around six receptionist views. Decisions D14–D26	277	—
5	PR #17 — items A1, A3. Config layer over all ~20 env vars; Anthropic + OpenAI providers with prompt caching and per-call usage; D8-a re-pinned to the Claude 5 family	277 → 325	2.0
6	PR #18 — items A2, A4. Engine and session scope, pooler-safe by default; the composition root and the five database port implementations it needed; a process-level worker pool so the webhook still answers before classification; alembic aligned to the same connection path	325 → 375	1.5
7	Items A5, A6 — Track A complete. Conversation and task continuity in the message path; the clarifying-turn budget honoured; classifications populated with measured telemetry; process() async; the v1 rules/destinations filer excised (D24); the outbound sender, and the channel credentials moved behind channels/, which emptied KNOWN_LEAKS and closed item 1.1	375 → 406	3.25
8	Items B1, B2, B3. Supabase project provisioned, migrated and stamped, with the channel_configs row tenant resolution raises without; migration 004 — RLS enabled and forced on all 20 tables behind a watcher_app role that cannot bypass it, a per-transaction tenant GUC carried by every adapter, and one narrow SELECT-only exception for the endpoint lookup that runs before a tenant is known; the container image, whose first build exposed a missing package data file and a removed Starlette API; deployed to Render, Frankfurt, and serving	406 → 417	2.75

Cumulative: 18 engineering days delivered. ~34.25 remaining.